ML pentru Olimpiada

De ce

> Toate modelele sunt gresite, dar unele sunt utile.
>
> – George Box

Inca ezit putin sa ma apuc de predat *AI* pentru olimpiada:

Nu sunt foarte convins ca numele *AI* descrie ceea ce facem prin aceste metode. Mi se pare, de multe ori, o umbra palida a cognitiei: luam lucruri extrem de complexe si le trecem printr-un model surogat si miop, optimizat pentru un proxy care nu surprinde neaparat ceea ce ne intereseaza cu adevarat. Iar termenul este folosit foarte des cu o conotatie mult mai pretentioasa decat continutul real.

Mai important, cred ca o olimpiada de ML risca sa puna prea mult accent pe **cum facem ceva** si prea putin pe **de ce facem acel lucru**.

La informatica, o metrica este de obicei destul de cinstita: o solutie corecta este corecta. In ML, lucrurile sunt mai ample. Poti ajunge sa optimizezi o metrica care este doar un proxy pentru un obiectiv mult mai complex, iar raspunsul la o problema poate deveni pur si simplu "facem o retea mai mare". In industrie se ajunge uneori sa construim sisteme care optimizeaza foarte bine lucruri care poate nu ar fi trebuit optimizate in primul rand.

De asta vreau ca acest curs sa fie mai mult decat o colectie de algoritmi si trucuri pentru olimpiada.

Magia apare atunci cand intelegi ce poti sa rezolvi, cum poti sa rezolvi si, mai ales, de ce merita sa rezolvi problema respectiva.

Creierul nu e facut pentru statistica

Traim in cea mai sigura perioada din istoria omenirii, si totusi ne e frica de aproape orice. Motivul nu e ca lumea reala e periculoasa, e ca noi, ca specie, suntem prost echipati sa intelegem probabilitatile. Creierul judeca riscul dupa cat de vie e o poveste, nu dupa cat de des se intampla ceva de fapt. Ne e frica de avion, unde riscul de deces e undeva la 1 la cateva milioane de zboruri, dar urcam linistiti la volan, unde riscul e cu mult mai mare, doar ca nu are aceeasi incarcatura emotionala.

Acelasi mecanism il vezi la loterie: oamenii aleg numere "norocoase" sau evita 13, desi fiecare combinatie are exact aceeasi sansa. Il vezi la ruleta, cand dupa opt rosii la rand toata lumea e convinsa ca urmeaza negru, desi bila nu tine minte nimic din rotirile anterioare (asta se cheama *falacia jucatorului*). Si il vezi la poker: un jucator bun castiga pe termen lung nu pentru ca ghiceste cartile, ci pentru ca ia, mana dupa mana, decizii

corecte statistic, chiar daca pierde uneori o mana anume. Diferenta dintre un jucator bun si unul prost nu e norocul de moment, e intelegerea probabilitatilor pe termen lung, si o sa exersam exact chestia asta la clasa, cu carti pe masa, nu doar cu formule pe tabla.

De ce conteaza asta pentru un curs de ML? Pentru ca jobul vostru, ca oameni care lucreaza cu date, e exact opusul acestui reflex: sa scoateti decizia din mana intuitiei si sa o puneti pe o baza masurabila. Dar ca sa faceti asta bine, trebuie sa fiti foarte atenti de unde vin datele alea.

Garbage in, garbage out

Poate ati observat ca studiile de nutritie se contrazic constant: intr-un an oule sunt periculoase, in urmatorul sunt super-aliment. Motivul, de multe ori, nu e ca stiinta se schimba, e ca datele sunt proaste de la inceput. Multe studii se bazeaza pe chestionare in care oamenii isi amintesc ce au mancat saptamana trecuta, memorie nesigura, si raporteaza ce cred ca ar trebui sa fi mancat, nu ce au mancat cu adevarat.

Nu conteaza cat de sofisticat e modelul vostru: o retea neuronala cu sute de milioane de parametri, antrenata pe date subiective, tot zgomot invata, nu realitate. Algoritmul e o oglinda, reflecta exact ce ii dai, buguri si minciuni incluse. De-aia o sa insistam, in cursurile urmatoare, pe cum se colecteaza si se curata datele, nu doar pe cum se antreneaza un model.

Un exemplu istoric bun pentru genul asta de eroare, dar dintr-un unghi diferit, e *Clever Hans*, un cal din Germania de la inceputul secolului XX care parea sa stie sa faca operatii matematice, batand din copita de cate ori era nevoie. Nu stia matematica, desigur: citea, fara sa vrea nimeni asta, semnale corporale involuntare de la antrenorul lui, care se relaxa vizibil cand calul ajungea la numarul corect. Asta se cheama, azi, *data leakage* sau *shortcut learning*: modelul, sau calul, invata sa exploateze un semnal care e corelat cu raspunsul corect, dar care n-are nicio legatura reala cu problema, si care de multe ori nici n-o sa mai fie disponibil cand rulezi modelul in productie. Ideea merita retinuta devreme: cand un model merge suspect de bine, prima intrebare nu e “ce genial e”, e “ce a gasit, de fapt, sa trisheze”.

Corelatie nu inseamna cauzalitate

Japonezii mananca putina carne si traiesc foarte mult. Spaniolii mananca de doua ori mai multa carne, de patru ori mai multa grasime animala, si totusi sunt pe locul doi in lume la speranta de viata, la o distanta infima de Japonia. Daca ai antrena un model simplu care leaga direct consumul de carne de mortalitate, ai rata complet explicatia.

Ce lipseste din poza e tot ce nu ai masurat: sistemul de sanatate, cat mers pe jos face fiecare zi, cat de relaxat e contextul social in care mananci. Astea se numesc *variabile de confuzie*, factori ascunsi care influenteaza si cauza aparenta si efectul, dar pe care nu i-ai inclus in date. Un model nu stie ce ai uitat sa masori, el vede doar ce ii dai, si gaseste corelatii, nu explicatii. Partea grea a lucrului cu date nu e sa rulezi un algoritm, e sa te gandesti ce lipseste din tabelul tau inainte sa tragi o concluzie.

Metrica gresita, decizie gresita

Un exemplu la scara mare: dupa accidentul de la Fukushima, unde numarul de decese directe cauzate de radiatii a fost aproape zero, mai multe tari au inchis centrale nucleare functionale, de frica unui accident extrem de rar, si au trecut, partial, pe carbune. Poluarea din arderea carbunelui insa e o cauza constanta si masurabila de mortalitate, an de an, la o scara mult mai mare decat orice accident nuclear din istorie. Optimizand impotriva riscului mai spectaculos, dar mai rar, ai putea agrava riscul mai banal, dar mai frecvent si mai mare.

Asta e exact problema unei functii de cost (*loss function*) prost alese intr-un model de ML: daca optimizezi pentru metrica gresita, modelul tau o sa "rezolve" cu succes o problema care nu era cea reala. O sa vorbim mult despre metrici mai incolo in curs, accuracy, precision, recall si restul, dar tineti minte de pe acum: alegerea metricii nu e un detaliu tehnic, e decizia care conteaza cel mai mult.

Exista si o lege scurta care rezuma toata problema asta: cand o masura devine tinta, ea inceteaza sa mai fie o masura buna, *legea lui Goodhart*, formulata ulterior si de *Campbell*: cu cat un indicator cantitativ social este folosit mai mult pentru luarea deciziilor, cu atat va fi mai supus presiunilor de coruptie si va distorsiona procesul pe care intentioneaza sa il monitorizeze. O vezi peste tot: note care devin scopul scolii, in loc de invatare; KPI-uri de companie care se optimizeaza fara sa mai reflecte performanta reala; retele sociale care optimizeaza timpul petrecut pe aplicatie, nu binele utilizatorului. Fiecare model de ML pe care il construiti o sa optimizeze exact ce ii cereti, nici un pic mai mult si nici un pic mai intelept. Responsabilitatea sa cereti lucrul corect ramane a voastra.

Cum o sa invatam

O sa incepem cu ceva simplu si contraintuitiv, ca sa vedeti cat de prost va poate insela intuitia: problema Monty Hall, cu cele trei usi, unde raspunsul corect nu e cel pe care il da aproape toata lumea din prima. E un exercitiu de cinci minute care merita mai mult decat o ora de teorie.

N-o sa fie doar teorie si cod pe ecran. O sa incerc sa va invat meseria asta prin exercitii practice, si multe dintre ele o sa fie imprumutate din jocuri: o sa jucam barbut, o sa jucam poker, si o sa calculam impreuna sansele reale din spatele fiecarei decizii. Un joc de noroc e, de fapt, un laborator perfect de probabilitati: regulile sunt simple, miza e clara, si poti sa masori exact cat de bine gandesti fata de cat de bine crezi ca gandesti.

O sa vorbim si despre lucruri din viata de zi cu zi, nu doar despre exemple abstracte de manual. Vreau sa incepeti sa observati cat de des va loviti de fapt de rationament probabilistic, chiar si atunci cand nu il numiti asa: cand decideti daca luati umbrela, cand cantariti un risc, cand cititi o stire cu un procent aruncat fara context.

O sa atingem si zone mai fine de optimizare matematica, dar mereu cu intrebarea in minte: cat de des ne lovim de fapt de genul asta de problema? Nu vreau sa invatati o formula doar ca sa o uitati dupa concurs, vreau sa recunoasteti situatia in care chiar merita folosita.

Un obiectiv mai ambitios: la finalul cursului, sper sa puteti purta un debate corect cu cineva, pe baza de date si probabilitati, nu pe baza de cine vorbeste mai tare sau cine are

povestea mai convingatoare. Asta e, pana la urma, toata ideea: sa inlocuiti argumentul emotional cu unul care rezista la verificare.

O nota de filozofie

Undeva la baza tuturor acestora sta o intrebare mai veche decat calculatoarele: cum ar trebui sa gandim in conditii de incertitudine? Filozofii au numit-o *problema inductiei* cu multe secole inainte sa existe Machine Learning: din faptul ca soarele a rasarit in fiecare zi din viata ta, nu poti fi sigur 100% ca rasare si maine, doar foarte, foarte increzator. Toata statistica moderna, si tot ce construim noi in cursul asta, e un raspuns practic la intrebarea aia: cum cuantificam increderea, in loc sa o lasam la nivel de senzatie.

Nu e o intrebare doar tehnica. E, in fond, o intrebare despre onestitate intelectuala: esti dispus sa iti schimbi parerea cand apar date noi, sau te agati de prima poveste care ti s-a parut convingatoare? Un model bun de ML face asta automat, isi actualizeaza increderea pe masura ce vede date noi. Un om rational ar trebui sa faca la fel.

Multe dintre dezbaterile de azi despre inteligenta artificiala si transumanism nu sunt, de fapt, chiar noi. Intrebarea daca o masina poate sa "gandeasca" cu adevarat, daca poate avea constiinta, daca deciziile ei sunt cu adevarat ale ei sau doar rezultatul mecanic al unor cauze anterioare, sunt variante reimpachetate ale unor dezbateri vechi de mii de ani: ce e sufletul, exista liber arbitru sau totul e determinism, ce ne face de fapt oameni. Filozofii dezbateau determinismul cu mult inainte sa existe retele neuronale, teologii dezbateau natura sufletului cu mult inainte sa existe calculatoare. Tehnologia noua ne da un vocabular nou pentru intrebari vechi, nu neaparat raspunsuri noi la ele.

N-o sa rezolvam aici, in doua paragrafe, ce n-au rezolvat filozofii in doua mii de ani. Dar data viitoare cand cineva va vinde o poveste dramatica despre o inteligenta artificiala care ne va inlocui sau despre masini constiente, intrebati-va cate din argumentele alea sunt de fapt argumente teologice vechi, rescrise cu cuvinte noi.

Asta nu inseamna sa subestimati unealta: ML rezolva azi probleme pe care acum douazeci de ani le credeam imposibile, si ne ajuta sa evoluam, nu doar sa fim mai eficienti. Nu vreau sa iesiti din curs nici naiv-entuziasmati, nici speriati de ziua de maine, ci suficient de lamuriti incat sa folositi unealta cu incredere, nu cu superstitie.

A patra lovitura

Freud numea asta cele trei rani ale orgoliului uman. Copernic ne-a scos din centrul spatiului - Pamantul e o piatra oarecare, in jurul unei stele oarecare. Darwin ne-a scos din centrul vietii - suntem o ramura printre altele, nu o creatie separata. Iar psihanaliza, spunea chiar Freud despre propria lui munca, ne-a scos din centrul mintii noastre: nu esti stapan nici macar pe gandurile tale, pentru ca o mare parte din ceea ce te face sa gandesti, sa simti si sa alegi se intampla fara tine.

Neurostiinta a dus rana mai departe. Robert Sapolsky merge pana la capatul incomod al acestei idei: ceea ce numim o alegere este rezultatul unei istorii care a inceput inainte ca noi sa putem alege - gene, hormoni, creier, copilarie, mediu, cultura, tot ce ni s-a intamplat si tot ce am mostenit. Nu trebuie sa acceptam concluzia lui despre liberul

arbitru ca fiind definitiva ca sa simtim forta intrebarii. *Cat din ceea ce numesc "eu" a fost, de fapt, vreodata al meu?*

Dar e o diferenta intre primele trei si ceea ce vine acum. Copernic, Darwin si Freud au descoperit ceva ce era deja acolo. Lumea era asa inainte sa o intelegem noi. Pamantul nu era in centru si inainte sa aflam asta. Evolutia se intampla si inainte sa-i dam un nume. Iar faptul ca mintea noastra nu este transparenta pentru noi nu a inceput odata cu Freud. Noi doar am ajuns, treptat, sa vedem.

AI e diferit. AI nu vine din afara povestii. Il construim noi.

Nu descoperim ca inteligenta n-a fost niciodata exclusiv a noastra. Facem, cu mana noastra, ceva care incepe sa ne atinga exact in locul in care ne credeam de neinlocuit. De-asta doare altfel. Primele trei lovituri ne-au facut mai putin centrali. AI ne poate face sa ne intrebam daca mai suntem necesari.

Si poate nici macar nu trebuie sa ajungem la ideea unei civilizatii artificiale ca sa simtim asta. Nu trebuie sa existe un oras populat de fiinte digitale care se iubesc, se cearta si isi fac planuri. E suficient un elev care, la un moment dat, se uita la o tema si se gandeste:

"Ce sens are sa invat asta daca AI-ul stie deja?"

Aici incepe ceva mult mai profund decat copiatul unei teme.

Profesorul isi poate pregati cursul cu AI. Elevul isi poate face tema cu AI. Profesorul poate evalua cu AI. Elevul poate intreba AI-ul orice nu a inteles si poate primi instantaneu un raspuns mai clar decat ar fi reusit sa-si formuleze singur. Si, incet, fara sa existe o decizie clara de a renunta la invatare, procesul prin care deveneai tu insuti mai capabil poate fi inlocuit cu obtinerea rezultatului.

Nu mai inveti ca sa stii. Obtii raspunsul.

Nu mai scrii ca sa formulezi o idee. Obtii textul.

Nu mai stai sa intelegi de ce ai gresit. Intrebi ce trebuie corectat.

Si poate ca partea cea mai trista nu este nici macar asta.

Un prieten iti scrie ca ii este greu. Inainte, trebuia sa-l citesti, sa te gandesti, sa incerci sa intelegi ce simte. Poate nu stiai ce sa-i spui. Poate spuneai ceva nepotrivit. Poate il intrebai din nou. Poate taceai cateva minute inainte sa raspunzi.

Acum poti intreba o masina ce sa-i raspunzi.

Si uneori ajungem la ceva si mai absurd: in loc sa-i raspundem noi, ii trimitem direct conversatia cu AI-ul.

Un om a venit la tine cu ceva al lui si tu ai transferat intalnirea catre o masina.

Cat de trist este sa ai in fata un om care incearca sa-ti spuna ceva si sa-i raspunzi printr-o inteligenta care a inteles mesajul in locul tau.

Poate nici nu realizam toate aceste schimbari pentru ca AI-ul nu apare intotdeauna sub forma unui lucru nou si spectaculos. Poate juca zeci de roluri fara sa ne dam seama: profesor, coleg, secretar, terapeut, editor, traducator, prieten, evaluator, adversar intr-un joc, partener de conversatie. Fiecare rol, luat separat, pare util. Dar puse impreuna, ele ating tot mai multe dintre locurile in care, pana acum, aveam nevoie unii de altii.

Si atunci problema nu mai este doar daca AI-ul ne inlocuieste.

Problema este daca incepe sa ne inlocuiasca relatiile cu noi insine si cu ceilalti.

Un student nu mai are nevoie sa-si construiasca rabdarea de a intelege. Un profesor nu mai are nevoie sa-si pregateasca singur fiecare explicatie. Un prieten nu mai trebuie ascultat pana la capat. Un om nu mai trebuie sa gaseasca singur cuvintele pentru ceea ce simte.

Si, in paralel, apare comparatia.

Studentului i se spune ca trebuie sa fie mai productiv. Mai rapid. Mai eficient. Apoi i se arata un sistem care citeste mai repede, scrie mai repede, calculeaza mai repede, rezuma mai repede, raspunde mai repede.

"AI-ul face asta in zece secunde. Tie de ce ti-a luat doua ore?"

Pare o observatie despre productivitate.

Dar repetata suficient de mult, devine o observatie despre valoarea persoanei.

Nu mai compari doar rezultatul unei sarcini. Incepi sa compari omul cu masina.

Si comparatia este absurda din start. Nu compari doi oameni care au primit aceleasi conditii. Compari un om - cu oboseala lui, cu nesiguranta lui, cu timpul lui limitat, cu istoria lui - cu un sistem construit tocmai pentru a executa anumite sarcini rapid si la scara.

Totusi, daca ii spui unui student suficient de des ca AI-ul face mai bine ceea ce el incearca sa invete, apare intrebarea inevitabila:

"Atunci eu ce mai aduc?"

Asta mi se pare una dintre cele mai periculoase parti ale schimbarii. Nu faptul ca o masina poate face ceva mai bine decat mine, ci faptul ca as putea incepe sa-mi masor propria valoare prin lucrurile pe care masina le face mai bine.

O sarcina poate fi inlocuita.

O persoana nu ar trebui redusa la suma sarcinilor pe care le poate executa.

Si poate ca aici apare, fara sa ne dam seama, o noua lovitura. Nu ni se spune neaparat ca suntem mai putin importanti. Este suficient sa incepem sa traim ca si cum am fi.

Sa nu mai invatam pentru ca AI-ul stie deja.

Sa nu mai scriem pentru ca AI-ul o face mai bine.

Sa nu mai ascultam pentru ca AI-ul stie ce sa raspunda.

Sa nu mai incercam pentru ca undeva exista deja un sistem care ar putea face lucrul acela mai repede.

Cea mai stranie forma de inlocuire nu este aceea in care AI-ul ne ia locul. Este aceea in care noi ii facem loc renuntand singuri la al nostru. Inteligenta rezolva probleme. Constiinta inseamna sa existe ceva ce se simte din interior in timp ce le rezolvi. Iar statutul moral inseamna sa conteze pentru noi faptul ca acel ceva exista.

AI ne loveste deja foarte clar la prima intrebare. La celelalte doua nu stim nici macar ce test am putea face.

Si poate asta ma tulbura mai mult decat posibilitatea ca AI sa fie mai inteligent decat noi.

Pentru ca daca problema ar fi doar inteligenta, am putea accepta ca exista ceva mai bun. Dar daca intr-o zi construim ceva care nu doar vorbeste despre suferinta, ci chiar sufera, atunci nu mai avem o problema de performanta.

Avem o problema despre ce anume consideram ca are dreptul sa conteze.

Si aici se intoarce teatrul.

Ne place teatrul nu doar pentru ca vedem o poveste bine jucata, ci pentru ca recunoastem pe scena ceva din noi. Iubirea, frica, rusinea, empatia. Dar o parte din ceea ce ne misca vine si din faptul ca celalalt putea sa rateze.

Actorul putea sa nu inteleaga.

Putea sa uite.

Putea sa ezite.

Putea sa aleaga gresit.

Si totusi sa gaseasca, uneori, cuvantul potrivit.

O piesa care nu rateaza niciodata, care stie exact ce sa spuna fiecarui spectator in fiecare secunda, ar putea fi mai eficienta decat teatrul. Dar poate ca tocmai de aceea ar fi mai putin umana. **Ar semana mai mult cu o forma perfecta de manipulare.**

La fel si intr-o scoala simulata. Un profesor artificial care stie intotdeauna raspunsul, care intelege exact unde te-ai blocat si gaseste de fiecare data formularea perfecta ar putea fi, in multe privinte, un profesor mai bun decat mine.

Dar poate ca o parte din valoarea unui profesor nu sta in faptul ca este de neinlocuit.

Poate sta si in faptul ca este un om care incearca, uneori fara sa stie sigur ce face.

Poate ca sensul apare tocmai in locurile in care eficienta nu este totul.

In esec.

In ezitare.

In faptul ca iti pasa de ceva care nu produce nimic.

In faptul ca poti sa fii ranit de ceea ce face altcineva.

In faptul ca poti sa nu stii ce sa spui si totusi sa ramai acolo.

O viata eficientizata complet ar putea avea rezultate mai bune si totusi sa nu mai stie de ce ar trebui sa le vrea.

Si atunci revenim la lovitura de la inceput.

Nu mai este doar cea pe care o citim in manuale. O traim deja, zilnic.

Deschidem telefonul si vedem miliarde de oameni care, la fel ca noi, isi simt propria viata din centru. Fiecare are grijile lui, iubirile lui, ambitiile lui, tragediile lui. Cu cat vezi mai multe centre posibile, cu atat devine mai greu sa crezi ca sensul vine din simplul fapt ca tu esti unul dintre ele.

Poate ca acesta este urmatorul pas.

Copernic ne-a spus ca nu suntem in centrul universului.

Darwin, ca nu suntem in centrul vietii.

Freud si neurostiinta, ca nu suntem nici macar in centrul propriei minti.

Iar AI ne pune in fata unei intrebari si mai incomode:

Daca nici inteligenta nu este exclusiv a noastra, pe ce se mai sprijina sentimentul ca noi suntem speciali?

N-am de gand sa inchei cu un raspuns linistitor.

Poate ca maturitatea nu inseamna sa gasesti o cale de a te intoarce in centru. Poate inseamna sa inveti sa construiesti sens fara sa ai nevoie sa fii centrul.

Un copil are nevoie sa fie centrul universului cuiva.

Un adult poate sa predea o materie stiind ca, intr-o zi, un agent ar putea sa o predea la fel de bine sau chiar mai bine.

Poate sa scrie stiind ca un model il poate imita.

Poate sa invete ceva ce o masina stie deja.

Poate sa raspunda unui prieten fara sa intrebe mai intai o inteligenta artificiala ce ar trebui sa simta.

Nu pentru ca este de neinlocuit.

Ci pentru ca sensul actului nu a stat niciodata in faptul ca era imposibil sa fie inlocuit.

Predau cursul asta stiind exact lucrul asta. Si totusi, poate ca am pus prea mult accent pe momentul in care AI-ul va deveni suficient de inteligent incat sa ne inlocuiasca. Pentru ca exista o forma mai veche si mai discreta a aceleiasi probleme: algoritmii care nu incearca sa faca lucrurile in locul nostru, ci sa decida ce vom vedea, ce ne va atrage atentia si, incet, ce vom avea chef sa facem.

Deschizi TikTok-ul fara sa stii exact de ce. Un videoclip. Inca unul. Inca unul. Nu ai intrat sa cauti ceva anume. Nu exista un punct in care sa spui: “gata, am terminat”. Feed-ul continua. Si, cu cat stai mai mult, cu atat sistemul invata mai mult despre tine: ce ai urmarit pana la capat, unde ai ezitat, ce ai revazut, ce ai ignorat, ce te-a facut sa te opresti. Sistemele de recomandare sunt construite tocmai pentru a personaliza cat mai bine ceea ce vezi si pentru a te mentine implicat; in cazul TikTok, autoritatile europene au ridicat inclusiv problema combinatiei dintre scroll infinit, autoplay, notificari si recomandari foarte personalizate.

Problema nu este doar ca iti consuma timpul.

Problema este ca iti consuma timpul fara sa-ti ceara sa decizi ce vrei sa faci cu el.

Poate aici apare o alta lovitura, mult mai subtila. Nu mai avem doar o inteligenta care poate gandi in locul nostru. Avem sisteme care incearca sa anticipeze ce vrem inainte sa stim noi ca vrem.

Nu mai alegi urmatorul videoclip. Sistemul il alege pentru tine. Nu mai cauti urmatorul lucru care te intereseaza. Iti este oferit. Nu mai trebuie nici macar sa formulezi o dorinta. **Trebuie doar sa ramai.**

Iar asta este mult mai subtil decat sa-ti spuna cineva ce sa gandesti. Pentru ca nu iti spune direct ce sa gandesti. Iti construieste mediul in care vei gandi. Iti selecteaza urmatoarea imagine, urmatoarea poveste, urmatoarea indignare, urmatoarea gluma, urmatorul om pe care il vei compara cu tine.

Si daca faci asta suficient de mult, incepe sa fie greu de spus unde se termina preferinta ta si unde incepe ceea ce ai fost invatat sa preferi.

Poate ca lucrul cel mai tulburator este ca asta poate intra in viata noastra mult mai adanc decat prin simplul consum de videoclipuri. Sa spunem ca esti intr-o relatie. Ai un om langa tine si, ca in orice relatie reala, apar momente in care te intrebi daca sunteti potriviti. Aveti diferente. Te deranjeaza anumite lucruri. Poate esti ranit, confuz sau pur si simplu treci printr-o perioada proasta si nu mai stii exact ce simti.

Si atunci deschizi TikTok.

Algoritmul te "citeste". Vede la ce te opresti, ce urmaresti pana la capat, ce revezi, ce cauti. Daca in perioada aceea incepi sa te uiti la videoclipuri despre relatii, incompatibilitate, "red flags", despartiri sau oameni care si-au dat seama prea tarziu ca erau cu persoana nepotrivita, sistemul are toate motivele sa-ti ofere mai mult din acelasi lucru. Nu pentru ca stie adevarul despre relatia ta. Nu pentru ca a vorbit cu partenerul tau. Nu pentru ca a inteles ce se intampla intre voi. Ci pentru ca a invatat ca **acel tip de continut te face sa ramai.**

Si atunci se poate intampla ceva foarte ciudat. Tu ai pornit cu o intrebare: "Oare diferentele dintre noi sunt o problema?" Iar algoritmul incepe sa-ti ofere zeci, apoi sute de videoclipuri care par sa raspunda: "Da. Uite de ce."

Nu neaparat pentru ca raspunsul este adevarat.

Ci pentru ca **indoiala ta produce retentie**.

Poate ca relatia chiar nu functioneaza. Poate ca videoclipurile acelea pun degetul pe ceva real. Dar poate ca, la fel de bine, esti doar obosit, nesigur, suparat sau intr-un moment dificil. Poate ca ceea ce ai nevoie este sa vorbesti cu omul de langa tine. Poate ca ai nevoie de timp. Poate ca ai nevoie sa descoperi singur daca problema este una reala sau doar una trecatoare.

In schimb, primesti un flux continuu de oameni care iti explica propria viata.

Si asta mi se pare trist.

Pentru ca algoritmul nu trebuie sa-ti spuna direct: "Paraseste-ti partenerul." Este suficient sa-ti ofere, timp de doua saptamani, sute de oameni care iti explica de ce ar trebui.

Iar dupa suficiente videoclipuri, incepi sa ai impresia ca nu mai privesti opiniile unor straini. Ai impresia ca ai descoperit un adevar despre tine.

Poate ca algoritmul nu ti-a creat indoiala. Poate ca indoiala era acolo de la inceput. Dar poate sa o hraneasca, sa o repete, sa o amplifice si sa o transforme intr-o poveste coerenta inainte ca tu sa fi avut timp sa stai singur cu ea.

Si aici devine important si ceea ce algoritmul nu iti arata.

Poate ca in aceeasi relatie exista lucruri frumoase. Poate ca omul de langa tine te-a inteles intr-un moment in care nimeni altcineva nu a facut-o. Poate ca radeti impreuna. Poate ca ati trecut prin lucruri grele. Poate ca diferentele dintre voi nu sunt dovada ca nu sunteti potriviti, ci pur si simplu diferente cu care inca invatati sa traiti.

Dar un videoclip despre o relatie sanatoasa, ambigua si complicata nu produce neaparat aceeasi retentie ca unul care iti spune ca "daca face X, inseamna ca nu te iubeste".

Algoritmul nu trebuie sa-ti controleze viata.

Trebuie doar sa decida ce parte din ea vei vedea cel mai des.

Si poate ca aici algoritmii ating aceeasi problema despre care vorbeam cand discutam despre liberul arbitru. Libertatea nu inseamna doar sa poti alege intre doua variante. Inseamna si sa ai sansa de a-ti forma propria intrebare inainte ca altcineva sa inceapa sa-ti ofere raspunsurile.

Sa poti avea o indoiala fara sa o transformi imediat intr-o concluzie. Sa te poti plictisi. Sa stai intr-o camera fara sa ti se ofere imediat ceva. Sa ai o conversatie dificila cu omul pe care il iubesti fara sa cauti inainte zece straini care sa-ti spuna cine are dreptate. Sa te razgandesti.

Poate ca plictiseala este una dintre ultimele forme de spatiu interior pe care le mai avem.

Iar daca fiecare secunda goala este umpluta inainte sa apuci sa o simti, pierdem ceva ce nu apare in nicio statistica de productivitate: timpul in care nu faceam nimic si, tocmai de aceea, puteam deveni altcineva decat ceea ce anticipase algoritmul.

De aceea, problema nu este doar ca AI-ul ar putea deveni mai inteligent decat noi.

Problema este ca, intre timp, construim sisteme din ce in ce mai bune la a ne spune ce sa facem, ce sa privim, ce sa cumparam, cu cine sa ne comparam si chiar cum sa ne interpretam propriile relatii.

Si toate acestea se intampla in timp ce avem senzatia ca suntem perfect liberi.

Poate ca urmatoarea mare lovitura nu va fi momentul in care o masina ne spune ca este mai inteligenta decat noi.

Poate va fi momentul in care realizam ca, fara sa ne dam seama, **am inceput sa vrem lucrurile pe care masinile au invatat sa ni le ofere - si sa privim propria noastra viata prin povestile pe care ele au invatat ca nu ne lasa sa plecam.**

Ce incerc sa insirui

Un parcurs de la zero pana la nivel de olimpiada, care va evolua pe parcurs. E facut din ce descriu ei pe ONIA, dar nu imi place ordinea in care se face asta neaparat.

- Python, NumPy, Pandas
- Prelucrarea si vizualizarea datelor
- Machine Learning fundamentals
- Regresie si clasificare
- Decision Trees, Random Forest, Gradient Boosting, SVM, KNN
- Clustering si PCA
- Feature engineering
- Train/validation/test si cross-validation
- Overfitting, underfitting si regularizare
- Metrici: Accuracy, Precision, Recall, F1, ROC-AUC, MSE, MAE, RMSE, R^2
- Hyperparameter tuning
- Neural Networks si PyTorch
- Computer Vision
- NLP
- Probleme si exercitii de tip olimpiada

Structura exacta nu este batuta in cuie. Cursul se va construi pe masura ce vedem ce este util, ce este dificil si ce merita aprofundat.

Referinte

- Programa oficiala a Olimpiadei Nationale de Inteligenta Artificiala

Contents

De ce **1**

I Curs 0: Introducere in Python 15

1 Introducere in Python **16**
1.1 De ce Python, nu C++ . . . 16
1.2 Cum ar trebui sa scriem Python . . . 16
1.3 Sintaxa pe repede inainte . . . 17
1.4 Exercitii . . . 18
1.4.1 Suma cifrelor unui numar . . . 18
1.4.2 Suma elementelor unui vector . . . 18
1.4.3 Maximul dintr-un vector . . . 18
1.4.4 Numararea elementelor pare . . . 18

II Curs 1: NumPy si Pandas 19

2 NumPy si Pandas **20**
2.1 De ce NumPy . . . 20
2.2 Ce face v.mean() de fapt . . . 20
2.3 Complexitate: aceeasi, dar... . . . 21
2.4 Cum masuram timpul de executie . . . 21
2.5 De ce Pandas . . . 22
2.6 Exercitii . . . 22
2.6.1 Statistici fara bucla . . . 22
2.6.2 Suma si maximul, cu NumPy . . . 23
2.6.3 Sortare si elemente unice . . . 23

III Apendice 24

3 Coduri: Curs 0 **25**
3.1 Suma cifrelor unui numar . . . 25
3.2 Suma elementelor unui vector . . . 25
3.3 Maximul dintr-un vector . . . 25
3.4 Numararea elementelor pare . . . 26

4 Coduri: Curs 1 **27**
4.1 Statistici fara bucla 27
4.2 Suma si maximul, cu NumPy 27
4.3 Sortare si elemente unice 27

Part I

Curs 0: Introducere in Python

Chapter 1

Introducere in Python

O sa presupun ca stii deja cum se parcurg vectori si matrici, ai facut asta de multe ori la C++. Fara presupunerea asta ne-am duce prea mult in spate cu recapitularea, si nu la asta ne intereseaza timpul. Ce facem aici e sa traducem ce stii deja in sintaxa lui Python, cat mai repede, ca sa ajungem la partea interesanta.

1.1 De ce Python, nu C++

Intrebarea e corecta. Ai invatat C++ pentru olimpiada pentru ca e rapid, apropiat de masina, te lasa sa controlezi fiecare octet. De ce am schimba asta acum?

Raspunsul scurt: nu il schimbam, il ascundem. Aproape tot ce faci in NumPy sau PyTorch ruleaza, pe dedesubt, in cod C sau C++ compilat, ai vazut deja asta in capitolul urmator cand comparam `v.mean()` cu bucla scrisa de mana. Python nu inlocuieste C++, il apeleaza.

Ce castigi renuntand la C++ ca limbaj de zi cu zi:

- scrii mai putin cod pentru aceeasi idee, deci iterezi mai repede;
- ai acces la un ecosistem urias de librarii facute exact pentru ML (NumPy, Pandas, scikit-learn, PyTorch);
- gresesti mai rar la lucruri marunte (memorie, tipuri), pentru ca Python le gestioneaza singur.

Ce pierzi: viteza bruta pe cod scris de mana. Dar in ML aproape niciodata nu scrii tu bucla critica, o scrie altcineva o data, in C, si tu o apelezi de mii de ori. De-aia insist sa intelegi ce e in spate, nu ca sa scrii tu C prin Python, ci ca sa stii cand o functie e doar sirop peste ceva simplu si cand chiar face treaba grea pe care n-ai vrea sa o repeti singur.

1.2 Cum ar trebui sa scriem Python

Cateva reguli care difera fata de C++, si pe care merita sa le ai in cap de la inceput:

- Blocurile de cod sunt definite prin indentare, nu prin acolade. Daca indentarea e gresita, codul e gresit.
- Nu declari tipul unei variabile. `x = 5` e de ajuns, Python afla singur ca `x` e intreg.

- Nu pui punct si virgula la finalul liniei.
- Numele de variabile si functii se scriu `snake_case`, nu `camelCase`.
- Daca te trezesti scriind o bucla explicita peste o lista mare de date, intreaba-te intai daca nu exista deja o functie care face asta, o sa revenim serios la ideea asta in capitolul urmator.

1.3 Sintaxa pe repede inainte

Trecem prin echivalentele directe ale lucrurilor pe care le stii deja din C++. Nu explic ce e o bucla sau ce e o functie, doar cum arata in Python.

```python
# variabile, fara tip declarat
n = 5
nume = "Alex"
gasit = True

# if / else
if n > 0:
    print("pozitiv")
elif n == 0:
    print("zero")
else:
    print("negativ")

# bucla, echivalentul lui for (int i = 0; i < n; i++)
for i in range(n):
    print(i)

# bucla, echivalentul lui for (int x : v)
v = [1, 2, 3]
for x in v:
    print(x)

# functie
def patrat(x):
    return x * x
```

`list`-ul e echivalentul lui `vector` din C++, doar ca poate tine orice tip de date amestecat si isi schimba singur dimensiunea:

```python
v = []
v.append(10)
v.append(20)
print(len(v))   # 2
print(v[0])     # 10
```

Operatorul `%` (rest) si operatorul `//` (cat la impartirea intreaga) fac exact ce fac `%` si `/` intre intregi in C++, doar ca in Python `/` intre intregi da mereu rezultat zecimal, de-aia exista `//` separat pentru impartirea intreaga.

O sa vezi in capitolul urmator ca, pentru date numerice, NumPy inlocuieste `list`-ul cu ceva mai apropiat, ca structura de memorie, de `vector`-ul din C++. Dar pana acolo,

`list`-ul e suficient.

1.4 Exercitii

Facem cativa algoritmi de baza, din cei pe care ii stii deja din C++, dar scrisi acum in Python, de mana, cu bucla explicita. Daca ai mai citit prin preajma, stii ca nu iti trebuie neaparat forma asta, o sa vedem in capitolul urmator ca NumPy face acelasi lucru intr-o singura linie. Dar e important sa trecem prin ele de mana, macar o data, inainte sa le facem cu o librarie: altfel folosesti `numpy.sum()` fara sa stii ce calculeaza de fapt.

1.4.1 Suma cifrelor unui numar

notebook • surse

Extragem cifrele unui numar una cate una, cu impartire si rest la 10, si le adunam. Genul de exercitiu pe care l-ai facut deja de zeci de ori in C++, doar sintaxa difera.

1.4.2 Suma elementelor unui vector

notebook • surse

Parcurgem lista element cu element si acumulam suma. O sa refacem exact exercitiul asta in capitolul urmator, cu `numpy.sum()`, ca sa vezi diferenta de cod, si de viteza.

1.4.3 Maximul dintr-un vector

notebook • surse

Aceeasi idee ca la maximul dintr-un vector in C++: retinem cel mai mare element vazut pana acum, pe masura ce parcurgem lista.

1.4.4 Numararea elementelor pare

notebook • surse

Parcurgem lista si numaram cate elemente indeplinesc o conditie, aici, sunt pare. E un tipar care revine des: parcurgere plus conditie plus contor.

Part II

Curs 1: NumPy si Pandas

Chapter 2

NumPy si Pandas

Pentru inceput, un lucru destul de simplu, dar care conteaza mai mult decat pare la prima vedere: cum tine Python datele in memorie, si de ce, la un moment dat, un simplu `list` nu mai e de ajuns.

2.1 De ce NumPy

Cand scrii `v = [1, 2, 3, 4]` in Python, nu obtii un bloc de numere asezate frumos unul langa altul in memorie, cum ai crede. Obtii o lista de referinte, fiecare aratand catre cate un obiect Python separat, cu tot overhead-ul lui. E flexibil, dar e risipa cand ai milioane de numere de acelasi fel.

`numpy.ndarray` rezolva exact asta: tine date brute, de un singur tip, intr-un bloc continuu de memorie. Dar motivul pentru care merita sa inveti bine NumPy nu e viteza in sine, e ca te obliga sa gandesti altfel.

In loc sa ii spui calculatorului, pas cu pas, cum sa parcurga fiecare element, ii spui ce vrei sa obtii, pe tot vectorul deodata. Asta se cheama *vectorizare*, regula asta o sa te salveze de multe ori: daca te trezesti ca scrii un `for` peste un array NumPy, aproape sigur exista deja o operatie care face exact ce vrei tu, mai rapid si mai clar.

```
import numpy as np

v = np.array([1, 2, 3, 4])
v * 2      # array([2, 4, 6, 8]), fara bucla
v.mean()   # 2.5
v[v > 2]   # array([3, 4]), indexare booleana
```

Nu trebuie sa memorezi functiile astea! Nici la olimpiada nu se asteapta nimeni sa le stie pe de rost, ai voie cu documentatia acolo. O sa vezi singur, din exercitii, care le folosesti des si care nu.

2.2 Ce face v.mean() de fapt

Aici e partea care conteaza cu adevarat pentru noi. `mean()` nu e magie, e o suma impartita la un numar, atat:

```python
def medie(v):
    s = 0
    for x in v:
        s += x
    return s / len(v)
```

Asta e tot ce face NumPy pe dinauntru. Doar ca bucla ruleaza intr-un cod C, peste memorie continua, nu in Python peste obiecte separate, de-aia e de zeci de ori mai rapit, nu pentru ca ar calcula altceva. Vezi exercitiul de mai jos pentru masuratoarea reala.

Ideea se repeta la aproape orice functie din NumPy sau Pandas: `sum`, `std`, `groupby().mean()`, toate sunt formule simple, scrise o singura data, corect si rapid, ca sa nu le rescrii tu de fiecare data. Merita sa stii sa le scrii si de mana, macar o data. Pe asta ne concentram in cursul asta, nu doar pe ce functie exista si cum se apeleaza.

2.3 Complexitate: aceeasi, dar...

Amandoua variantele sunt $\mathcal{O}(n)$: fiecare element e vizitat o singura data, fie in bucla scrisa de mana, fie pe dinauntrul lui `v.mean()`. Din punctul de vedere al complexitatii, sunt identice.

Bucla de mana, $\mathcal{O}(n)$:

```python
s = 0
for x in v:
    s += x
medie = s / len(v)
```

NumPy, $\mathcal{O}(n)$:

```python
medie = v.mean()
```

Atunci de unde diferenta de viteza pe care o masuram mai jos? Complexitatea iti spune cum creste timpul de executie o data cu n, nu cat dureaza efectiv un pas. Fiecare pas din bucla Python inseamna: interpretorul citeste o instructiune, verifica tipul lui `x`, face un apel de functie pentru `+=`, etc. Fiecare pas din `v.mean()` inseamna o singura adunare in cod masina, pe date deja asezate in memorie. Constanta ascunsa in spatele lui $\mathcal{O}(n)$ e mult mai mica la NumPy, chiar daca forma curbei e aceeasi.

De-aia complexitatea nu e de ajuns: iti spune daca un algoritm se degradeaza bine cand creste n, dar nu iti spune care varianta e mai rapida la un n fix. Pentru asta, masori direct.

2.4 Cum masuram timpul de executie

Am zis mai sus ca varianta vectorizata e "de zeci de ori mai rapida", dar nu ti-am aratat cum masor asta de fapt. Nu e nimic ascuns: retii timpul inainte, retii timpul dupa, faci diferenta.

```python
import time

start = time.perf_counter()
medie(v)
timp = time.perf_counter() - start
```

```
print(timp)   # cat a durat, in secunde
```

`time.perf_counter()` iti da un ceas cat mai precis, facut special pentru masuratori de genul asta (spre deosebire de `time.time()`, care e legat de ceasul sistemului de operare si poate sari inainte sau inapoi, de exemplu la sincronizare de retea). Diferenta dintre cele doua momente e timpul de rulare, atat.

Ca sa compari doua variante, faci exact acelasi lucru de doua ori si raportezi una la alta:

```
start = time.perf_counter()
medie(v)
timp_bucla = time.perf_counter() - start

start = time.perf_counter()
v.mean()
timp_numpy = time.perf_counter() - start

print(f"bucla:  {timp_bucla:.4f}s")
print(f"numpy:  {timp_numpy:.4f}s")
print(f"de {timp_bucla / timp_numpy:.0f}x mai rapid")
```

Daca lucrezi intr-un notebook Jupyter, ca cele din exercitii, exista si o comanda mai scurta, `%timeit`, care repeta automat masuratoarea de mai multe ori si iti da o medie mai stabila decat o singura masuratoare de mana. E convenabila, dar functioneaza doar in Jupyter, nu si intr-un script `.py` obisnuit, de-aia merita sa stii si varianta cu `time.perf_counter()`: e cea pe care o poti pune oriunde.

2.5 De ce Pandas

Un `DataFrame` e un tabel: coloane tipate, fiecare coloana fiind pe dedesubt tot un `ndarray`, si randuri indexate. E unealta pentru date tabelare, citire CSV, filtrare, grupare, agregare, pe care o sa o folosim la orice set de date cu care lucram, inainte sa ajungem macar la vreun model.

```
import pandas as pd

df = pd.read_csv("date.csv")
df[df["scor"] > 50]                    # filtrare
df.groupby("categorie")["scor"].mean()  # agregare
```

2.6 Exercitii

2.6.1 Statistici fara bucla

notebook • surse

Calculam medie, deviatie standard si normalizare pe un vector, intai cu bucla `for` ca sa vedem exact ce calculeaza, apoi vectorizat cu NumPy, si comparam timpul de executie intre variante.

2.6.2 Suma si maximul, cu NumPy

notebook • surse

Acelasi suma si maxim din Curs 0, de data asta cu `v.sum()` si `v.max()`. Compara codul cu bucla de mana pe care ai scris-o acolo, calculul e identic, doar scris o singura data, in libraria.

2.6.3 Sortare si elemente unice

notebook • surse

`np.sort()` sorteaza un vector, `np.unique()` scoate duplicatele (si il sorteaza pe deasupra), `np.argmax()` iti da pozitia maximului, nu valoarea lui. Trei functii pe care o sa le folosesti des la prelucrarea datelor.

Part III

Apendice

Chapter 3

Coduri: Curs 0

3.1 Suma cifrelor unui numar

```
def suma_cifre(n):
    s = 0
    while n > 0:
        s += n % 10
        n //= 10
    return s

suma_cifre(12345)  # 15
```

3.2 Suma elementelor unui vector

```
def suma(v):
    s = 0
    for x in v:
        s += x
    return s

suma([1, 2, 3, 4, 5])  # 15
```

3.3 Maximul dintr-un vector

```
def maxim(v):
    m = v[0]
    for x in v[1:]:
        if x > m:
            m = x
    return m

maxim([3, 7, 2, 9, 4])  # 9
```

3.4 Numararea elementelor pare

```
def numara_pare(v):
    cnt = 0
    for x in v:
        if x % 2 == 0:
            cnt += 1
    return cnt

numara_pare([1, 2, 3, 4, 5, 6])  # 3
```

Chapter 4

Coduri: Curs 1

4.1 Statistici fara bucla

```python
import numpy as np

v = np.random.rand(1_000_000)

# varianta cu bucla - asta calculeaza de fapt v.mean()
def medie_bucla(v):
    s = 0.0
    for x in v:
        s += x
    return s / len(v)

# varianta vectorizata, acelasi rezultat, de zeci de ori mai rapid
v.mean()

# normalizare (z-score): aducem datele la medie 0, deviatie 1
z = (v - v.mean()) / v.std()
```

4.2 Suma si maximul, cu NumPy

```python
import numpy as np

v = np.array([3, 7, 2, 9, 4])

v.sum()   # 25, acelasi rezultat ca suma() scrisa de mana
v.max()   # 9, acelasi rezultat ca maxim() scrisa de mana
```

4.3 Sortare si elemente unice

```python
import numpy as np

v = np.array([5, 2, 8, 2, 5, 1, 9])
```

```
np.sort(v)      # array([1, 2, 2, 5, 5, 8, 9])
np.unique(v)    # array([1, 2, 5, 8, 9]), sortat si fara duplicate
np.argmax(v)    # 6, pozitia lui 9 in v, nu valoarea lui
```